

Laboratorio 1 : Patrones de Diseño Creacionales

El objetivo de este laboratorio es adquirir experiencia práctica en la aplicación de patrones de diseño de la categoría “Creacionales”, utilizados ampliamente en el desarrollo de software.

Cada estudiante, de forma individual, debe enfocarse en uno de los siguientes patrones de diseño (entre paréntesis se especifica el número de capítulo respectivo en el libro *Software Architecture Design Patterns in Java* de Partha Kuchana)

- Factory Method (Cap 10)
- Singleton (Cap 11)
- Abstract Factory (Cap 12)
- Prototype (Cap 13)

La asignación de patrones de diseño por estudiante se especifica en el Anexo 1. La sección 1 describe el contexto, reto técnico y evidencia necesaria para cada patrón. La sección 2 describe los entregables a entregar.

Valor en la nota: El laboratorio 1 es parte del componente Laboratorios – 15% de la nota final. Durante el curso de realizarán 3 laboratorios en total – 5% cada laboratorio

1. Patrones a utilizar

1.1 Factory Method

Contexto: Una aplicación necesita enviar alertas a los usuarios. Inicialmente solo envía SMS, pero el negocio exige añadir Email y notificaciones Push sin modificar el código que dispara la alerta.

Reto técnico:

1. Crear una interfaz Notificacion con el método enviar().
2. Implementar clases concretas (NotificacionSMS, NotificacionEmail).
3. Implementar una clase creadora abstracta NotificacionFactory con un método crearNotificacion().

Evidencia: Un programa principal que, basado en un archivo de configuración o entrada del usuario, instancie la fábrica correcta y envíe el mensaje sin usar un bloque if-else gigante para decidir la clase concreta.

1.2 Singleton

Contexto: Una aplicación empresarial necesita leer un archivo de configuración (puerto del servidor, URL de la base de datos, tiempo de timeout) al iniciar. Esta configuración debe ser accesible desde cualquier parte del código, pero cargar el archivo múltiples veces afectaría el rendimiento y podría causar inconsistencias si los datos cambian en memoria.

Reto técnico:

Crear una clase ConfiguracionSistema.

Hacer el constructor private para evitar instanciación externa.

Implementar un método estático getInstance() que gestione la creación perezosa (lazy initialization) y asegure que siempre devuelva la misma dirección de memoria.

Añadir atributos como urlBase y puerto con sus respectivos getters.

Evidencia: Un programa donde se soliciten dos instancias de ConfiguracionSistema en diferentes clases y se demuestre, mediante una comparación de identidad (instance1 == instance2), que ambas variables apuntan exactamente al mismo objeto.

1.3 Abstract Factory.

Contexto: Se está diseñando una herramienta que debe ejecutarse tanto en Windows como en macOS. Cada sistema tiene su propia estética para los componentes. Si el usuario está en Windows, todos los botones y ventanas deben ser estilo Windows; si está en macOS, deben ser estilo Mac. No se deben mezclar componentes de distintas familias.

Reto técnico:

Definir interfaces para los productos: Boton y Ventana.

Crear implementaciones concretas: BotonWindows, BotonMac, VentanaWindows, VentanaMac.

Crear una interfaz GUIFactory con los métodos crearBoton() y crearVentana().

Implementar las fábricas concretas: WindowsFactory y MacFactory.

Evidencia: Una clase Aplicacion que reciba una GUIFactory como parámetro en su constructor. El estudiante debe ejecutar el programa dos veces, pasando una fábrica distinta cada vez, y mostrar cómo la aplicación configura toda su interfaz correctamente sin conocer las clases concretas.

1.3 Prototype.

Contexto: En un videojuego, crear un "Enemigo Jefe" requiere una carga pesada de recursos (texturas, sonidos, configuración de IA compleja). A veces, el diseñador de niveles necesita colocar 10 copias del mismo jefe en una habitación, pero cada copia puede tener una pequeña variación (como el color o la posición inicial).

Reto técnico:

1. Crear una interfaz o clase abstracta Enemigo que implemente la interfaz Cloneable de Java (o defina su propio método clonar()).
2. Implementar una clase concreta EnemigoJefe con atributos pesados y un estado interno.
3. Implementar el método clonar() asegurando que se realice una copia profunda o superficial según convenga.
4. Crear un RegistroDePrototipos (opcional) que almacene los modelos base para ser clonados.

Evidencia: El programa debe crear un objeto original "Jefe" y luego generar 5 copias mediante clonación. El estudiante debe modificar un atributo (como la posición) en un clon y demostrar que el objeto original permanece intacto, probando que la clonación fue exitosa.

2. Entregables

Un documento en formato PDF con:

- Diagrama de Clases UML: Realizado en una herramienta como Draw.io o Lucidchart o PlantUML/PlantText que refleje su implementación.
- Justificación del Patrón: ¿Por qué este patrón mejora la mantenibilidad en comparación con una solución procedimental?
- Capturas de Pantalla: Ejecución del código en el IDE IntelliJ
- Enlace al repositorio público en Github. Este repositorio se evolucionará en futuros laboratorios

3. Fecha de entrega

Viernes 10 de abril de 2026, antes de las 11:59 pm

4. Anexo: asignación de patrones por estudiante

Juan Jose Alfaro Alfaro	Factory Method
Leonardo Jose Araya Cordero	Abstract Factory
Luis Daniel Barboza Alfaro	Singleton
Nicole Bou Espinoza	Prototype
Wesley Alberto Campos Raith	Prototype
Jose Mario Castro Cruz	Factory Method
Gabriel Castro Salazar	Singleton
Dana Gabriela Chavarria Corrales	Abstract Factory
Bryan Hernandez Gutierrez	Singleton
Anyelina Hernandez Villalobos	Abstract Factory
Tzu Rue Hsu Fu	Prototype
Joseph Ivan Monge Monge	Prototype
Justin Jamal Morris Bains	Singleton
Alexander Murillo Sancho	Factory Method
Michael Josep Oviedo Meza	Singleton
Dayana Rojas Campos	Prototype
Diego Josue Rojas Castro	Singleton
Jose Ignacio Rosich Gonzalez	Factory Method
Kenneth Rojas Jimenez	Abstract Factory
Alexander Salazar Rojas	Prototype
Kendall Antonio Sanchez Ramirez	Singleton
Steven Jesus Sancho Ramirez	Abstract Factory
Jose Andres Villegas Murillo	Prototype